Practical 1

AIM: Design a Simple Rational Agent by defining PEAS for Vacuum Cleaner Environment and Autonomous Taxi. Classify environments (fully/partially observable, deterministic/stochastic, episodic/sequential). Implement a simple table-driven agent in Python.(VACCUM CLEANER)

CODE:

```
class TableDrivenAgent:
    def __init__(self):
        self.table = {
            (('A', 'Dirty'),): 'Suck',
            (('A', 'Clean'),): 'Right',
            (('B', 'Dirty'),): 'Suck',
            (('B', 'Clean'),): 'Left',

            (('A', 'Clean'), ('B', 'Dirty')): 'Suck',
            (('A', 'Dirty'), ('A', 'Clean')): 'Right'
        }

        self.percepts = []

    def perceive(self, percept):
        self.percepts.append(percept)
        percept_sequence = tuple(self.percepts)

        action = self.table.get(
            percept_sequence,
            "No Action Found"
        )

        return action

agent = TableDrivenAgent()
print(agent.perceive(('A', 'Dirty')))

agent = TableDrivenAgent()
print(agent.perceive(('A', 'Clean')))

agent = TableDrivenAgent()
print(agent.perceive(('B', 'Dirty')))

agent = TableDrivenAgent()
print(agent.perceive(('B', 'Clean')))

```

OUTPUT:

```
Suck
Right
Suck
Left
PS C:\Users\ayesha\OneDrive\Documents\CODE\AI SEM 5>
```

Practical 2

AIM: Given an initial configuration of the 8-puzzle and a goal configuration, write a Python program to find the shortest sequence of moves to reach the goal state using Breadth-First Search (BFS).

CODE:

```
from collections import deque

def get_neighbors(state):
    neighbors = []

    # Find the blank (0)
    blank = state.index(0)
    row = blank // 3
    col = blank % 3

    # Possible moves: Up, Down, Left, Right
    moves = {
        "Up": (-1, 0),
        "Down": (1, 0),
        "Left": (0, -1),
        "Right": (0, 1)
    }

    for move, (dr, dc) in moves.items():
        new_row = row + dr
        new_col = col + dc

        if 0 <= new_row < 3 and 0 <= new_col < 3:
            new_blank = new_row * 3 + new_col

            new_state = list(state)
            new_state[blank], new_state[new_blank] = (
                new_state[new_blank],
                new_state[blank]
            )

            neighbors.append((tuple(new_state), move))

    return neighbors

def bfs(initial, goal):
    queue = deque([(initial, [])])
    visited = set()

    while queue:
        current_state, path = queue.popleft()

        if current_state == goal:
            return path

        if current_state in visited:
            continue

        visited.add(current_state)

        for next_state, move in get_neighbors(current_state):
            if next_state not in visited:
                queue.append((next_state, path + [move]))

    return None

# Initial state
```

```
initial_state = (
    1, 2, 3,
    4, 0, 5,
    6, 7, 8
)

# Goal state
goal_state = (
    1, 2, 0,
    4, 5, 3,
    6, 7, 8
)

# Find solution
solution = bfs(initial_state, goal_state)

# Display output
if solution:
    print("Shortest sequence of moves:")
    print(" -> ".join(solution))
    print("Number of moves:", len(solution))
else:
    print("No solution found.")
```

OUTPUT:

```
PS C:\Users\ayesha\OneDrive\Documents\CODE\AI SEM 5> & C:\Users\ayesh
 "c:/Users/ayesha/OneDrive/Documents/CODE/AI SEM 5/p2.py"
Shortest sequence of moves:
Right -> Up
Number of moves: 2
PS C:\Users\ayesha\OneDrive\Documents\CODE\AI SEM 5>
```

Practical 3

AIM: Write a Python program to find the shortest path using **A* Search** with the **Manhattan Distance heuristic**.(8 PUZZLE)

CODE:

```
import heapq

def get_goal_positions(goal):
    pos = {}
    for i, val in enumerate(goal):
        pos[val] = (i // 3, i % 3)
    return pos

def manhattan(state, goal_pos):
    distance = 0

    for i, val in enumerate(state):
        if val == 0:
            continue

        x, y = i // 3, i % 3
```

```
        gx, gy = goal_pos[val]

        distance += abs(x - gx) + abs(y - gy)

    return distance

def get_neighbors(state):
    neighbors = []

    blank = state.index(0)
    x, y = blank // 3, blank % 3

    moves = [
        ("Up", -1, 0),
        ("Down", 1, 0),
        ("Left", 0, -1),
        ("Right", 0, 1)
    ]

    for move, dx, dy in moves:
        nx = x + dx
        ny = y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:
            new_blank = nx * 3 + ny

            new_state = list(state)

            new_state[blank], new_state[new_blank] = (
                new_state[new_blank],
                new_state[blank]
            )

            neighbors.append((tuple(new_state), move))

    return neighbors

def astar(initial, goal):

    goal_positions = get_goal_positions(goal)

    priority_queue = []

    h = manhattan(initial, goal_positions)

    heapq.heappush(
        priority_queue,
        (h, 0, initial, [])
    )

    visited = {}

    while priority_queue:

        f, g, state, path = heapq.heappop(priority_queue)

        if state == goal:
            return path

        if state in visited and visited[state] <= g:
            continue

        visited[state] = g

        for neighbor, move in get_neighbors(state):

            new_g = g + 1

            h = manhattan(
                neighbor,
```

```
                goal_positions
            )

            new_f = new_g + h

            heapq.heappush(
                priority_queue,
                (
                    new_f,
                    new_g,
                    neighbor,
                    path + [move]
                )
            )

    return None

initial_state = (
    1, 2, 3,
    4, 0, 5,
    6, 7, 8
)

goal_state = (
    1, 0, 3,
    4, 5, 6,
    2, 7, 8
)

solution = astar(
    initial_state,
    goal_state
)

if solution:

    print("A* Search Solution (Optimal Path):")

    print(" -> ".join(solution))

    print("Number of moves:", len(solution))

else:

    print("No solution found")
```

OUTPUT:

```
PS C:\Users\ayesha\OneDrive\Documents\CODE\AI SEM 5> & C:\Users\ayesha\AppData\Local\Programs\Python\Python312\python.exe
  "c:/Users/ayesha/OneDrive/Documents/CODE/AI SEM 5/p3.py"
A* Search Solution (Optimal Path):
Up -> Left -> Down -> Right -> Right -> Down -> Left -> Left -> Up -> Right -> Down -> Right -> Up -> Left -> Left -> Up
-> Right
Number of moves: 17
PS C:\Users\ayesha\OneDrive\Documents\CODE\AI SEM 5>
```

Practical 4

AIM: Given 2 water jugs of 4 litres and 3 litres, write a Python program to obtain exactly 2 litres in one jug using Depth First Search (DFS).

CODE:

```python
def dfs_water_jug(cap1, cap2, target):
    start = (0, 0)
    stack = [(start, [])]
    visited = set()

    def get_next_states(state):
        x, y = state
        next_states = []

        next_states.append(((cap1, y), "Fill Jug 1"))
        next_states.append(((x, cap2), "Fill Jug 2"))

        next_states.append(((0, y), "Empty Jug 1"))
        next_states.append(((x, 0), "Empty Jug 2"))

        # Pour Jug 1 -> Jug 2
        pour = min(x, cap2 - y)
        next_states.append(
            ((x - pour, y + pour),
             "Pour Jug 1 -> Jug 2")
        )

        # Pour Jug 2 -> Jug 1
        pour = min(y, cap1 - x)
        next_states.append(
            ((x + pour, y - pour),
             "Pour Jug 2 -> Jug 1")
        )

        return next_states

    while stack:
        state, path = stack.pop()

        if state in visited:
            continue

        visited.add(state)

        x, y = state

        if x == target or y == target:
            return path + [f"Reached ({x}, {y})"]

        for next_state, action in get_next_states(state):
            if next_state not in visited:
                stack.append(
                    (next_state, path + [action])
                )

    return None

cap1, cap2 = 4, 3
target = 2

result = dfs_water_jug(cap1, cap2, target)

if result:
    print("Solution found using DFS:")
```

```
    for step in result:
        print(step)
else:
    print("No solution found")
```

OUTPUT:

```
PS C:\Users\ayesha\OneDrive\Documents\CODE\AI SEM 5> & C:\Users\ayesha\AppData
 "c:/Users/ayesha/OneDrive/Documents/CODE/AI SEM 5/p4.py"
Solution found using DFS:
Fill Jug 2
Pour Jug 2 -> Jug 1
Fill Jug 2
Pour Jug 2 -> Jug 1
Reached (4, 2)
PS C:\Users\ayesha\OneDrive\Documents\CODE\AI SEM 5>
```

Practical 5

AIM: Classify environments as fully/partially observable, deterministic/non-deterministic, episodic/sequential, and implement a table-driven agent in Python.(TAXI AUTOMATION)

CODE:

```
class TableDrivenAgent:

    def __init__(self, table):
        self.table = table

    def perceive_and_act(self, percept):
        return self.table.get(
            percept,
            "STOP AND WAIT"
        )

decision_table = {

    ("road clear", "green", "no obstacle"):
        "MOVE_FORWARD",

    ("road clear", "red", "no obstacle"):
        "STOP",

    ("traffic", "green", "obstacle ahead"):
        "TURN_LEFT",

    ("road clear", "green", "obstacle ahead"):
        "TURN_LEFT",

    ("road clear", "green", "pedestrian detected"):
        "STOP",

    ("heavy traffic", "green", "no obstacle"):
        "CHANGE_LANE"
}
```

```
32.
33. agent = TableDrivenAgent(decision_table)
34.
35. percepts = [
36.
37.     ("road clear", "green", "no obstacle"),
38.
39.     ("road clear", "red", "no obstacle"),
40.
41.     ("road clear", "green", "pedestrian detected"),
42.
43.     ("unknown state", "green", "no obstacle")
44. ]
45.
46. for p in percepts:
47.
48.     action = agent.perceive_and_act(p)
49.
50.     print(f"Percept: {p} -> Action: {action}")
51.
52.
```

OUTPUT:



Practical 8

AIM: Write a python program to find the solution using Greedy Best-first search with the Manhattan heuristic.

CODE:

```python
import heapq
def get_goal_positions(goal):
    pos = {}

    for i, val in enumerate(goal):
        pos[val] = (i // 3, i % 3)

    return pos


def manhattan(state, goal_pos):
    distance = 0

    for i, val in enumerate(state):

        if val == 0:
            continue

        x, y = i // 3, i % 3
        gx, gy = goal_pos[val]

        distance += abs(x - gx) + abs(y - gy)

    return distance


def get_neighbors(state):

    neighbors = []

    blank = state.index(0)

    x = blank // 3
    y = blank % 3

    moves = [
        ("Up", -1, 0),
        ("Down", 1, 0),
        ("Left", 0, -1),
        ("Right", 0, 1)
    ]

    for move, dx, dy in moves:

        nx = x + dx
        ny = y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:

            new_blank = nx * 3 + ny

            new_state = list(state)

            new_state[blank], new_state[new_blank] = (
                new_state[new_blank],
                new_state[blank]
            )

            neighbors.append(
                (tuple(new_state), move)
```

```
            )

    return neighbors


def greedy_best_first(initial, goal):

    goal_pos = get_goal_positions(goal)

    pq = []

    heapq.heappush(
        pq,
        (manhattan(initial, goal_pos),
         initial,
         [])
    )

    visited = set()

    while pq:

        h, state, path = heapq.heappop(pq)

        if state == goal:
            return path

        if state in visited:
            continue

        visited.add(state)

        for neighbor, move in get_neighbors(state):

            if neighbor not in visited:

                heapq.heappush(
                    pq,
                    (
                        manhattan(neighbor, goal_pos),
                        neighbor,
                        path + [move]
                    )
                )

    return None


initial_state = (
    1, 2, 3,
    4, 0, 5,
    6, 7, 8
)

goal_state = (
    1, 0, 3,
    4, 5, 6,
    2, 7, 8
)


solution = greedy_best_first(
    initial_state,
    goal_state
)


if solution:

    print("Solution using greedy Best-First Search:")
```

```
131.
132.     print(" -> ".join(solution))
133.
134.     print("Number of moves:", len(solution))
135.
136. else:
137.
138.     print("No solution found")
139.
```

OUTPUT:

